

AUTOMATION

Push events into a running session with channels

Use channels to push messages, alerts, and webhooks into your Claude Code session from an MCP server. Forward CI results, chat messages, and monitoring events so Claude can react while you're away.

ⓘ Channels are in [research preview](#) and require Claude Code v2.1.80 or later. They require Anthropic authentication through [claude.ai](#) or a Console API key, and are not available on Amazon Bedrock, Google Vertex AI, or Microsoft Foundry. Team and Enterprise organizations must [explicitly enable them](#).

A channel is an MCP server that pushes events into your running Claude Code session, so Claude can react to things that happen while you're not at the terminal. Channels can be two-way: Claude reads the event and replies back through the same channel, like a chat bridge. Events only arrive while the session is open, so for an always-on setup you run Claude in a background process or persistent terminal.

Unlike integrations that spawn a fresh cloud session or wait to be polled, the event arrives in the session you already have open: see [how channels compare](#).

You install a channel as a plugin and configure it with your own credentials. Telegram, Discord, and iMessage are included in the research preview.

When Claude replies through a channel, you see the inbound message in your terminal but not the reply text. The terminal shows the tool call and a confirmation (like "sent"), and the actual reply appears on the other platform.

If you manage a Team, Enterprise, or Console organization, see [Enable channels for your organization](#). To build your own channel, see the [Channels reference](#).

Supported channels

Each supported channel is a plugin that requires [Bun](#). For a hands-on demo of the plugin flow before connecting a real platform, try the [fakechat quickstart](#).

View the full [Telegram plugin source](#).

1

Create a Telegram bot

Open **BotFather** in Telegram and send `/newbot`. Give it a display name and a unique username ending in `bot`. Copy the token BotFather returns.

2

Install the plugin

In Claude Code, run:

```
/plugin install telegram@claude-plugins-official
```

If Claude Code reports that the plugin is not found in any marketplace, your marketplace is either missing or outdated. Run `/plugin marketplace update claude-plugins-official` to refresh it, or `/plugin marketplace add anthropics/claude-plugins-official` if you haven't added it before. Then retry the install.

After installing, run `/reload-plugins` to activate the plugin's configure command.

3

Configure your token

Run the configure command with the token from BotFather:

```
/telegram:configure <token>
```

This saves it to `~/.claude/channels/telegram/.env`. You can also set `TELEGRAM_BOT_TOKEN` in your shell environment before launching Claude Code.



Restart with channels enabled

Exit Claude Code and restart with the channel flag. This starts the Telegram plugin, which begins polling for messages from your bot:

```
claude --channels plugin:telegram@claude-plugins-official
```



Pair your account

Open Telegram and send any message to your bot. The bot replies with a pairing code.

❗ If your bot doesn't respond, make sure Claude Code is running with `--channels` from the previous step. The bot can only reply while the channel is active.

Back in Claude Code, run:

```
/telegram:access pair <code>
```

Then lock down access so only your account can send messages:

```
/telegram:access policy allowlist
```

Discord

View the full [Discord plugin source](#).



Create a Discord bot

Go to the [Discord Developer Portal](#), click **New Application**, and name it. In the **Bot** section, create a username, then click **Reset Token** and copy the token.



Enable Message Content Intent

In your bot's settings, scroll to **Privileged Gateway Intents** and enable **Message Content Intent**.

③

Invite the bot to your server

Go to **OAuth2 > URL Generator**. Select the `bot` scope and enable these permissions:

- View Channels
- Send Messages
- Send Messages in Threads
- Read Message History
- Attach Files
- Add Reactions

Open the generated URL to add the bot to your server.

④

Install the plugin

In Claude Code, run:

```
/plugin install discord@claude-plugins-official
```

If Claude Code reports that the plugin is not found in any marketplace, your marketplace is either missing or outdated. Run `/plugin marketplace update claude-plugins-official` to refresh it, or `/plugin marketplace add anthropics/claude-plugins-official` if you haven't added it before. Then retry the install.

After installing, run `/reload-plugins` to activate the plugin's configure command.

5

Configure your token

Run the configure command with the bot token you copied:

```
/discord:configure <token>
```

This saves it to `~/.claude/channels/discord/.env`. You can also set `DISCORD_BOT_TOKEN` in your shell environment before launching Claude Code.

6

Restart with channels enabled

Exit Claude Code and restart with the channel flag. This connects the Discord plugin so your bot can receive and respond to messages:

```
claude --channels plugin:discord@claude-plugins-official
```

7

Pair your account

DM your bot on Discord. The bot replies with a pairing code.

❗ If your bot doesn't respond, make sure Claude Code is running with `--channels` from the previous step. The bot can only reply while the channel is active.

Back in Claude Code, run:

```
/discord:access pair <code>
```

Then lock down access so only your account can send messages:

```
/discord:access policy allowlist
```

View the full [iMessage plugin source](#). The iMessage channel reads your Messages database directly and sends replies through AppleScript. It requires macOS and needs no bot token or external service.

①

Grant Full Disk Access

The Messages database at `~/Library/Messages/chat.db` is protected by macOS. The first time the server reads it, macOS prompts for access: click **Allow**. The prompt names whichever app launched Bun, such as Terminal, iTerm, or your IDE.

If the prompt doesn't appear or you clicked Don't Allow, grant access manually under **System Settings > Privacy & Security > Full Disk Access** and add your terminal. Without this, the server exits immediately with `authorization denied`.

②

Install the plugin

In Claude Code, run:

```
/plugin install imessage@claude-plugins-official
```

If Claude Code reports that the plugin is not found in any marketplace, your marketplace is either missing or outdated. Run `/plugin marketplace update claude-plugins-official` to refresh it, or `/plugin marketplace add anthropics/claude-plugins-official` if you haven't added it before. Then retry the install.

③

Restart with channels enabled

Exit Claude Code and restart with the channel flag:

```
claude --channels plugin:imessage@claude-plugins-official
```

④

Text yourself

Open Messages on any device signed into your Apple ID and send a message to yourself. It

reaches Claude immediately: self-chat bypasses access control with no setup.

❗ The first reply Claude sends triggers a macOS Automation prompt asking if your terminal can control Messages. Click **OK**.

5

Allow other senders

By default, only your own messages pass through. To let another contact reach Claude, add their handle:

```
/imessage:access allow +15551234567
```

Handles are phone numbers in `+country` format or Apple ID emails like `user@example.com`.

You can also **build your own channel** for systems that don't have a plugin yet.

Quickstart

Fakechat is an officially supported demo channel that runs a chat UI on localhost, with nothing to authenticate and no external service to configure.

Once you install and enable fakechat, you can type in the browser and the message arrives in your Claude Code session. Claude replies, and the reply shows up back in the browser. After you've tested the fakechat interface, try out **Telegram**, **Discord**, or **iMessage**.

To try the fakechat demo, you'll need:

- Claude Code **installed and authenticated** with a claude.ai account or a Claude Console API key
- **Bun** installed. The pre-built channel plugins are Bun scripts. Check with `bun --version`; if that fails, **install Bun**.
- **Team, Enterprise, or managed Console org:** your admin must **enable channels** in managed settings

1

Install the fakechat channel plugin

Start a Claude Code session and run the install command:

```
/plugin install fakechat@claude-plugins-official
```

If Claude Code reports that the plugin is not found in any marketplace, your marketplace is either missing or outdated. Run `/plugin marketplace update claude-plugins-official` to refresh it, or `/plugin marketplace add anthropics/claude-plugins-official` if you haven't added it before. Then retry the install.

2 Restart with the channel enabled

Exit Claude Code, then restart with `--channels` and pass the fakechat plugin you installed:

```
claude --channels plugin:fakechat@claude-plugins-official
```

The fakechat server starts automatically.



You can pass several plugins to `--channels`, space-separated.

3 Push a message in

Open the fakechat UI at <http://localhost:8787> and type a message:

```
hey, what's in my working directory?
```

The message arrives in your Claude Code session as a `<channel source="fakechat">` event. Claude reads it, does the work, and calls fakechat's `reply` tool. The answer shows up in the chat UI.

If Claude hits a permission prompt while you're away from the terminal, the session pauses until you respond. Channel servers that declare the **permission relay capability** can forward these prompts to you so you can approve or deny remotely. For unattended use, **--dangerously-skip-permissions** bypasses prompts other than explicit ask rules, but only use it in environments you trust.

When you run channels in non-interactive mode with `-p`, tools that need terminal input, such as

multiple-choice questions and plan mode approval, are disabled so the session never stalls waiting for input.

Security

Every approved channel plugin maintains a sender allowlist: only IDs you've added can push messages, and everyone else is silently dropped.

Telegram and Discord bootstrap the list by pairing:

1. Find your bot in Telegram or Discord and send it any message
2. The bot replies with a pairing code
3. In your Claude Code session, approve the code when prompted
4. Your sender ID is added to the allowlist

iMessage works differently: texting yourself bypasses the gate automatically, and you add other contacts by handle with `/imessage:access allow`.

On top of that, you control which servers are enabled each session with `--channels`, and your organization controls availability with `channelsEnabled` on Claude.ai Team and Enterprise plans and on Console organizations that deploy managed settings.

Being in `.mcp.json` isn't enough to push messages: a server also has to be named in `--channels`.

The allowlist also gates permission relay if the channel declares it. Anyone who can reply through the channel can approve or deny tool use in your session, so only allowlist senders you trust with that authority.

Enterprise controls

Admins control availability through two managed settings that users cannot override. The default depends on how you authenticate:

- **Claude.ai Team and Enterprise:** channels are blocked until an Owner enables them.
- **Anthropic Console with API key authentication:** channels are permitted by default. You only need this setting if your organization deploys managed settings.

In all cases, no channel runs until a user opts it in for the session with `--channels`.

Setting	Purpose	When not configured
<code>channelsEnabled</code>	Master switch. Must be <code>true</code> for any channel to deliver messages. Set via the claude.ai Admin console toggle or directly in managed settings. Blocks all channels including the development flag when off.	claude.ai Team and Enterprise: channels blocked. Console: channels allowed unless your organization deploys managed settings, in which case channels are blocked until this key is set
<code>allowedChannelPlugins</code>	Which plugins can register once channels are enabled. Replaces the Anthropic-maintained list when set. Only applies when <code>channelsEnabled</code> is <code>true</code> .	Anthropic default list applies

Pro and Max users without an organization skip these checks entirely: channels are available and users opt in per session with `--channels`.

Enable channels for your organization

Enable channels for your organization from [claude.ai → Admin settings → Claude Code → Channels](#), which requires the Owner role, or by setting `channelsEnabled` to `true` in managed settings.

Once enabled, users in your organization can use `--channels` to opt channel servers into individual sessions. If the setting is disabled or unset, the MCP server still connects and its tools work, but channel messages won't arrive. A startup warning tells the user to have an admin enable the setting.

Restrict which channel plugins can run

By default, any plugin on the Anthropic-maintained allowlist can register as a channel. Admins on Team and Enterprise plans can replace that allowlist with their own by setting `allowedChannelPlugins` in managed settings. Use this to restrict which official plugins are allowed, approve channels from your own internal marketplace, or both. Each entry names a plugin and the marketplace it comes from:

```
{
  "channelsEnabled": true,
  "allowedChannelPlugins": [
    { "marketplace": "claude-plugins-official", "plugin":
"telegram" },
    { "marketplace": "claude-plugins-official", "plugin":
"discord" },
    { "marketplace": "acme-corp-plugins", "plugin": "internal-
alerts" }
  ]
}
```

When `allowedChannelPlugins` is set, it replaces the Anthropic allowlist entirely: only the listed plugins can register. Leave it unset to fall back to the default Anthropic allowlist. An empty array blocks all channel plugins from the allowlist, but `--dangerously-load-development-channels` can still bypass it for local testing. To block channels entirely including the development flag, leave `channelsEnabled` unset instead.

This setting requires `channelsEnabled: true`. If a user passes a plugin to `--channels` that isn't on your list, Claude Code starts normally but the channel doesn't register, and the startup notice explains that the plugin isn't on the organization's approved list.

Research preview

Channels are a research preview feature. Availability is rolling out gradually, and the `--channels` flag syntax and protocol contract may change based on feedback.

During the preview, `--channels` only accepts plugins from an Anthropic-maintained allowlist, or from your organization's allowlist if an admin has set `allowedChannelPlugins`. The channel plugins in `claude-plugins-official` are the default approved set. If you pass something that isn't on the effective allowlist, Claude Code starts normally but the channel doesn't register, and the startup notice tells you why.

To test a channel you're building, use `--dangerously-load-development-channels`. See [Test during the research preview](#) for information about testing custom channels that you build.

Report issues or feedback on the [Claude Code GitHub repository](#).

How channels compare

Several Claude Code features connect to systems outside the terminal, each suited to a different kind of work:

Feature	What it does	Good for
<u>Claude Code on the web</u>	Runs tasks in a fresh cloud sandbox, cloned from GitHub	Delegating self-contained async work you check on later
<u>Claude in Slack</u>	Spawns a web session from an @Claude mention in a channel or thread	Starting tasks directly from team conversation context
Standard <u>MCP server</u>	Claude queries it during a task; nothing is pushed to the session	Giving Claude on-demand access to read or query a system
<u>Remote Control</u>	You drive your local session from claude.ai or the Claude mobile app	Steering an in-progress session while away from your desk

Channels fill the gap in that list by pushing events from non-Claude sources into your already-running local session.

- **Chat bridge:** ask Claude something from your phone via Telegram, Discord, or iMessage, and the answer comes back in the same chat while the work runs on your machine against your real files.
- **Webhook receiver:** a webhook from CI, your error tracker, a deploy pipeline, or other external service arrives where Claude already has your files open and remembers what you were debugging.

Next steps

Once you have a channel running, explore these related features:

- **Build your own channel** for systems that don't have plugins yet
- **Remote Control** to drive a local session from your phone instead of forwarding events into it
- **Scheduled tasks** to poll on a timer instead of reacting to pushed events